CompSci234/NetSys210 Advanced Topics in Networking Assignment #2 Software Defined Networking with P4 and ONOS

Instructor: Cheng-Hsin Hsu
chsu@cs.nthu.edu.tw

University of California, Irvine
January 26, 2019

1 Objective

Software Defined Networking (SDN) is comprised of three layers: (i) application layer, where software programs leverage the services provided by the control layer, (ii) control layer, where a controller provides APIs, drivers, protocols, etc. to the application layer, and (iii) infrastructure layer, which contains programmable network devices. In this project, we will learn how these layers work together, and implement our own SDN programs.

2 Descriptions

The infrastructure layer could be based on ASIC chips with multiple pipelines. It may be reprogrammed to process incoming packets in various ways. For example, an ASIC chip can: (i) parse each incoming packet and drop the malicious ones or (ii) edit packets before forwarding them. The control layer collects some information from the infrastructure layer, including throughputs and forwarding tables. Such information is accessible to network administrators through APIs and web pages. Each application program receives the information and updates the flow tables in the infrastructure layers through the control layer. In this project, we will build software on ONOS controller and bmv2 switches, which are introduced below.

3 Software Tools

3.1 Environment/VM Setup

We have prepared a VM image at `https://tinyurl.com/ydgc68vb`. You may use VirtualBox to import the image; while using other hypervisors is also possible. The username is test and the password is compsci234. Once you login to the VM, you should see a `P4_testbed/` folder under your home directory. Please use that folder as the starting point of this assignment.

3.2 Bmv2

3.2.1 Description

Bmv2 stands for behavior model version 2. Its github repository is here [2]. The repository also contains the installation guide and some sample code. Feel free to install it on your own machine and try the sample codes. Bmv2 is a software switch written in C++. In bmv2 repository, there are several sample switches under targets/ folder. `Simple_switch` is a software switch that supports most features defined in P4 spec.

This switch employs thrift rpc for communicating with the controller, and is not a bad starting point for small-scale experiments. In this project, we will start from another software switch called `simple_switch_grpc`, which also supports p4runtime (that allows communications between controllers and P4 switches) and thrift. While `simple_switch_grpc` supports both thrift and grpc. It's not recommended to use both at the same time. Doing so may lead to synchronization problems related to reads/writes, because they implement some common apis. One way to get around this is to use p4runtime solely for writes and thrift solely for reads.

By using bmv2, you can turn your machine into a software switch. Combining with mininet, you can launch multiple software switches in virtual environments on a single machine. Although it's more convenient to conduct P4 experiments with bmv2, bear in mind that hardware vendors may not implement all P4 features. Hence, there is no guarantee that your code will run on real P4 switches.

3.2.2 Sample Usage

The simplest test for bmv2 is to feed your P4 program into it to launch a software switch.

```
       // compile p4 program
       p4c-bmv2 --json output.json input.p4
       // feed p4 program
       simple_switch -i 1@s1h1 -i 2@s1s2 -i 3@s1s3 --nanolog
ipc:///tmp/bm-1-log.ipc --device-id 1 sdn-bfr.json --log-file /tmp/p4s.s1.log
--log-flush --log-level trace
```

3.3 Mininet

3.3.1 Description

Mininet is a tool for network developers to run their experiments on a single workstation. It could be installed by apt-get on Ubuntu Linux, and through other package management systems (on other Linux distributions). Mininet has a manual page [3]. Please read it.

3.3.2 Sample Usage

Start the virtual network of mininet by

```
sudo mn
```

This gives you the default network topology. You may pass some arguments to customize your network topology. For example, topo, tree 3 leads to a tree topology with 3 levels. Other parameter and class usages can be found in the manual page.

4 Tasks

This assignment consists of two tasks: (i) P4 programming on bmv2 and (ii) SDN application programming on ONOS.

4.1 Task 1: Realizing IP Forwarding in P4

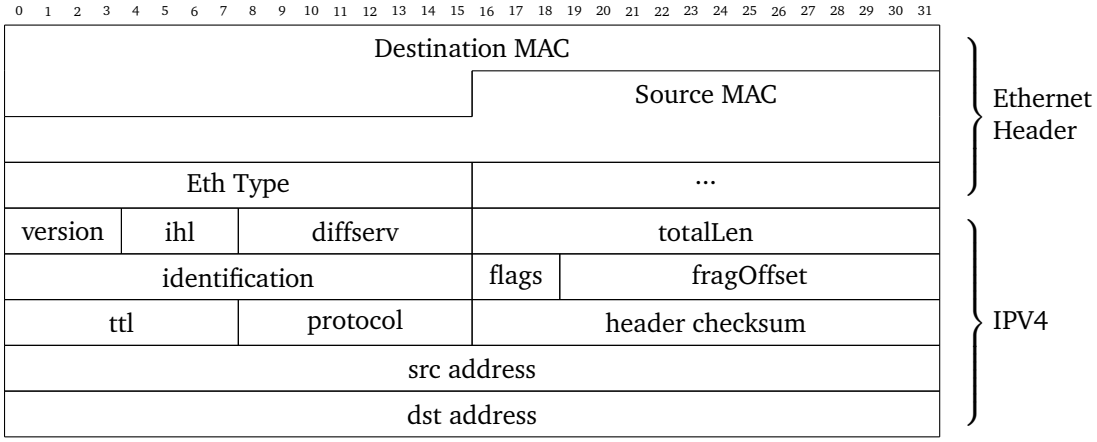
4.1.1 Purpose

In P4, packets are first parsed and passed to the ingress program. After that, they are sent to a deparser and then through the egress program to the output port. More details on P4 can be found [1]. Internet Protocol Version 4 (IPV4) is the most popular protocol in the Internet. In this task, we are going to implement a P4 program to *forward* IPV4 packets. Your P4 program should consist of four parts: (i) header, (ii) parser,

(iii) tables, and (iv) control. To complete this assignment, you need to finish all of them and figure out the commands for launching the software switches. You can find all the files you need under P4_testbed/. At the end, your switches should recognize IPV4 packets and support the *pingall* test in mininet. Note that each of the subtasks below discusses a file that needs to be modified by you.

4.1.2 Subtasks

1. p4src/task1.p4
   (a) Header
      i. You can follow the header format below to define IP header. Search for TODOs in the P4 file for additional hints.



   (b) Parser

   P4 parser is a Finite State Machine. It starts from *start* state and ends at *accept* state. We have to define the states and the transitions among states according to the header content. For example, the parser starts from parsing the Ethernet header with eth-type IPV4. The parser then jumps to IPV4 headers and returns to the ingress control once the parsing is done. In this task, you will implement the protocol layer and checksum calculation, at least. You are free to support other protocols or higher layer headers.

   (c) Match and Action Tables

   In P4, actions are declared as sequences of predefined functions. At the runtime, the table is used to select the actions associated with the table entries. In an action, you can modify metadata, drop packet, or clone the packet. There is also a table in P4 spec telling what you may do in an action. Some logic rules such as if-else, and, or, etc. are possible. However, please be advised that P4 still imposes certain limitations, e.g., recursions and loops are not supported.

```
action action_name(param1, param2, ......) {

    /* some statements.....*/

}
```

   Each table entry contains a match field and an action field. There are several match types. Examples are given in the table below, you can also find more details in the P4 spec. In addition, if you don't want to match anything, just leave the match field blank, which defines the default action. The action field may include multiple actions.

Type	Meaning
Exact	The field value is exactly the same as the table value.
lpm	Longest Prefix Match (lpm) matches with the longest subnet mask.
valid	If the header field is successfully parsed.

(d) Control

A packet is processed by a sequence of tables. You can define multiple control flows. However, we need only one control function here, which is the ingress control function. It's one of the imperative functions. In this function, you need to determine which port you should send the packet to and how to modify the packet. Details can be found in the P4 spec.

2. commands.txt

The default script run_demo.sh will launch a predefined mininet network topology according topo.txt in the same folder. Then it executes commands in commands.txt to each of the switches. You must edit this file to set up the flow tables of individual switches in your topology. Note that, you may want to prepare a different commands.txt file for different switches. As an example, we put a commands0.txt in the same folder. Nonetheless, feel free to design your own topology.

3. run_demo.sh and topo.py (optional)

These two files are scripts to run your program. Feel free to edit these files if you want. For example, if you need to run different commands on different switches, you may want to edit topo.py.

4. Verify your implementation. After you finish everything, your scripts should be able to:

(a) Launch the mininet.

(b) Form the topology as specified by topo.txt.

(c) Pass the pingall test in mininet.

4.2 Task2: Implementing IP Tunnel App on ONOS

4.2.1 Purpose

We use Open Networking Operating System (ONOS) as the controller to monitor and control all P4 switches. ONOS is an opensource project initiated by Open Networking Foundation (ONF). ONOS provides a user friendly environment for network administrators and developers to control SDN and create applications. We go with the developer version of ONOS. Hence, we get to build ONOS ourselves and implement our own ONOS applications.

4.2.2 Subtasks

1. Running ONOS Applications

(a) Build and Run ONOS

Please change your directory to the root of ONOS project and build ONOS with some applications using the following commands. For example, Proxy Arp intercepts ARP requests, and LLDP discovers links among switches.

```
cd $ONOS_ROOT
ONOS_APPS=proxyarp,hostprovider,lldpprovider ok clean
```

Note that you start the ONOS with several apps by putting them after ONOS_APPS. The ok command is an alias to run ONOS on your local machine. Please note that you may need to wait for 5-10 minutes if it's the first time you run ONOS.

(b) Activate ONOS Applications

After ONOS is running, launch another terminal and use the ONOS client to connect to ONOS. Then, activate bmv2 driver and p4 tutorial pipeconf. Last, list all active applications to make sure that they are running. The precise commands are given below.

```
onos localhost
app activate org.onosproject.drivers.bmv2
app activate org.onosproject.p4tutorial.pipeconf
apps -a -s
```

(Optional) By default, ONOS peridically checks the flow table (e.g., stats) on individual swtiches. You may change the interval to 5 seconds by the command below. This command is often used to obtain flow stats such as byte/packet counters more frequently. It also helps resolving the out-of-sync problems among the tables on the ONOS controller and switches.

```
cfg set org.onosproject.net.flow.impl.FlowRuleManager
    fallbackFlowPollFrequency 5
```

(c) Launch Mininet

To run Mininet, use the following command.

```
sudo -E mn --custom $BMV2_MN_PY --switch onosbmv2,pipeconf=p4-
    tutorial-pipeconf --controller remote,ip=127.0.0.1
```

The custom argument tells the mininet to execute bmv2.py on switches. The environment variable $BMV2_MN_PY refers to the location of bmv2 python script. You can echo it to see where it exactly is.

(d) Packet Forwarding

Now try to ping among hosts. You can use command pingall or h1 ping h2 under the mininet prompt. However, this should *not* work right now. It is because we only activate applications to collect link status. You have to activate another application for IP forwarding.

```
app activate org.onosproject.fwd
```

This is an application provided by ONOS. You can find the source code in onos/Apps/fwd/. This app implements dynamic IP forwarding. You can observe those flow rules inserted by this app through simple_switch_CLI or ONOS GUI.

2. IP Tunneling Application

We provide a semi-finished implementation of IP tunneling protocol called mytunnel.p4, which depends on p4-tutorial-pipeline. The app is located under onos/apps/p4-tutorial/, and there are two files that you need to check out:

p4-tutorial/pipeconf/src/main/resources/mytunnel.p4
p4-tutorial/mytunnel/src/main/java/org/onosproject/p4tutorial/mytunnel/MyTunnelApp.java

(a) Protocol Overview

We now build an application to replace fwd. In the previous exercise, we mentioned that proxyarp, lldpprovider, hostprovider help us to deal with ARP, LLDP packets and give us information about each host. We now build our own IP tunnel among hosts by inserting flow rules to the switches. In mytunnel app, we register a host event listener. This event listener is triggered whenever a host is found or disconnected. Here we only concentrate on the events of hosts found. When a host is found, we identify a shortest path among it and other hosts and insert flow rules into switches between each pair of them. In this way, switches know how to handle packets by inspecting the ipv4 dst_addr and src_addr.

(b) Steps

i. P4 (Pipeconf)

A. Check the onos/apps/p4-tutorial/ folder.

B. Please look for annotations TODO in mytunnel.p4, and complete the missing parts.

C. Remember to compile the p4 program using Unix make.

ii. App (MyTunnelApp)

A. Find internalListener in MyTunnelApp.java.

B. Implement the method provision tunnel; and other places annotated with TODO.

Once you complete and compile both the P4 and the ONOS app. You can try to ping among hosts. If you have any questions about the ONOS java classes, please check out their website [4].

iii. Testing Commands. To test your code, rerun onos, and use onos client to activate the required applications as follows.

```
onos localhost
app activate org.onosproject.drivers.bmv2
app activate org.onosproject.p4tutorial.pipeconf
app activate org.onosproject.p4tutorial.mytunnel
apps -a -s
```

Then, run the above mininet command for pinall test.

5 Submission Guidelines

The grades are given mainly based on your report. If your report fails to convince the instructor, you may be asked to give a live demo.

5.1 Report

Following questions should be answered in your report to verify your understanding of the concepts in this assignment. Please organize your materials into two sections.

- Briefly present your implementations for Task 1. Please show your topology and commands, along with your results as screenshots.
- Briefly present your implementations for Task 2. Please also include screenshots of your ping test.

In the report, you are encouraged to discuss your observations. Please submit your PDF and PDF only.

5.2 Late Submission Policy

- Late within 24 hours: 25% reduction in marks.
- Late within a week: 50% reduction in marks.
- After one week: Your submission will not be graded.

6 Grading Policy

- Task 1 (7%): Ping test should work. Please explain what modifications have been done to complete this task. Please include the key code snippets.
- Task 2 (8%): Ping test should work. Please explain what modifications have been done to complete this task. Please include the key code snippets.

7 Additional Notes

The VM image is based on an image provided by ONF.

References

[1] Slides of ONOS tutorial provided by ONF
https://docs.google.com/presentation/d/1yrL8dzC_6a5p382EZL4RhrMTAPk4YRGyO5tPJ3qXM6s/edit#slide=id.g2657fbf0c4_0_302

[2] Official Github repository of software switch, behavior model version 2 (bmv2).
https://github.com/p4lang/behavioral-model

[3] Mininet Python API Reference Manual
http://mininet.org/api/index.html

[4] ONOS API Manual
http://api.onosproject.org/1.13.2

[5] BIER Draft
https://datatracker.ietf.org/doc/rfc8279/

[6] ONOS dev Mail List
https://onosproject.org/contribute/mailing-lists/